PATRÓN DE COMPORTAMIENTO

State

Un objeto que cambia su comportamiento según su estado, sin llenarse de condicionales.

Ejemplo 1 · La entrada

Ejemplo 2 · State + Observer

Boletería del Estadio Metropolitano · Real Madrid vs Atlético de Madrid

EL PROBLEMA

Un objeto, muchos comportamientos

Entrada.pagar() sin State

```
if (estado === 'disponible')
  return 'Primero reserva';
if (estado === 'reservada')
  { estado = 'pagada'; return 'OK'; }
if (estado === 'pagada')
  return 'Ya está pagada';
if (estado === 'usada')
  return 'Ya se usó';
```

El mismo if en cada método

reservar(), pagar(), cancelar() y entrar() repiten la pregunta «¿en qué estado estoy?».

Un estado nuevo toca todo

Añadir «Reembolsada» obliga a editar cada método y a no olvidar ninguno.

Reglas mezcladas

Las transiciones quedan escondidas entre condicionales difíciles de leer.

LA IDEA

Cada estado es una clase

1

El contexto delega

La entrada no decide nada: pasa cada acción al objeto de su estado actual.

2

El estado decide

Cada clase concreta sabe qué acciones acepta, cuáles rechaza y qué responde.

3

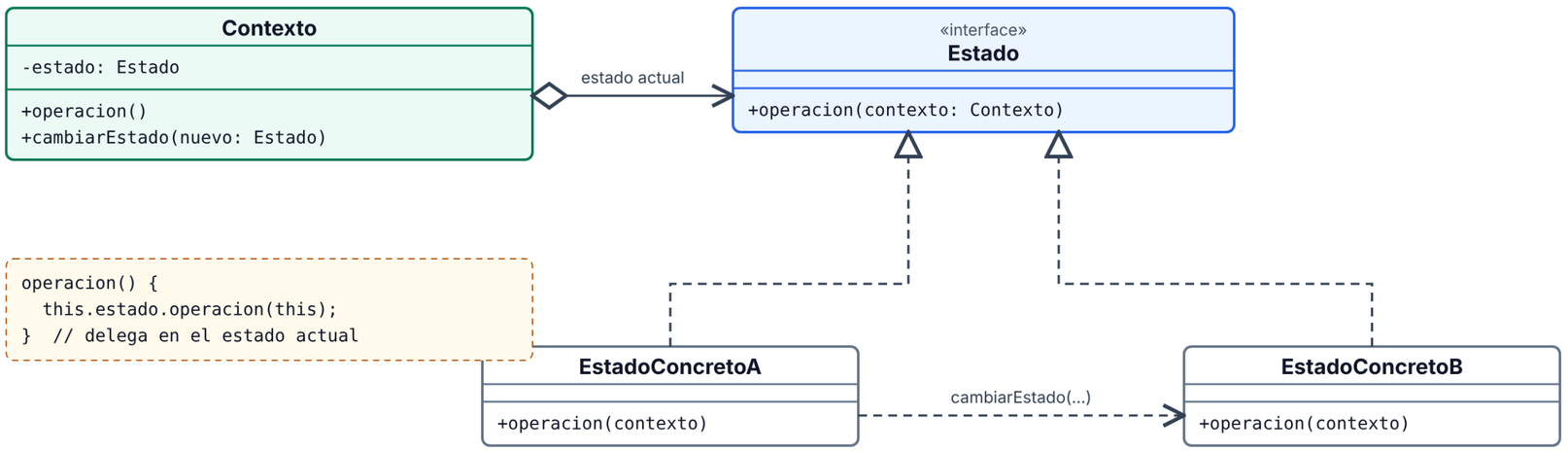
El estado cambia al contexto

Para pasar al siguiente estado, el estado actual le entrega uno nuevo a la entrada.

Cambiar el estado cambia la clase del objeto que responde.

DIAGRAMA DE CLASES

Estructura del patrón State



Contexto
Guarda el estado actual y le delega el trabajo.

Estado
Interfaz común con una operación por acción.

Estados concretos
Implementan su comportamiento y eligen el siguiente.

CUÁNDO USARLO

Ventajas y precauciones

Úsalo si...

- un objeto se comporta distinto según su estado,
- hay muchos estados que cambian con frecuencia,
- una clase está llena de condicionales sobre sus atributos.

Ganas

- Una clase por estado: cada una hace una sola cosa.
- Estados nuevos sin tocar los existentes.
- Un contexto simple, sin condicionales.

Cuidado

- Con dos o tres estados estables, es exagerado.
- Más clases y más archivos que un simple if.

EJEMPLO 1 · SOLO STATE

La vida de una entrada

Una entrada del derbi, Lateral Oeste, fila 12, asiento 8. Cuatro estados y cuatro acciones; cada estado acepta solo las que tienen sentido.

Disponible

Solo se puede reservar.

Reservada

Se paga o se cancela.

Pagada

Solo sirve para entrar; no hay devoluciones.

Usada

Estado final: el QR ya se escaneó.

Ruta de la demo: /patronState/simple · código en src/app/state-simple

EJEMPLO 1 · DIAGRAMA DE ESTADOS

Qué hace cada estado con cada acción

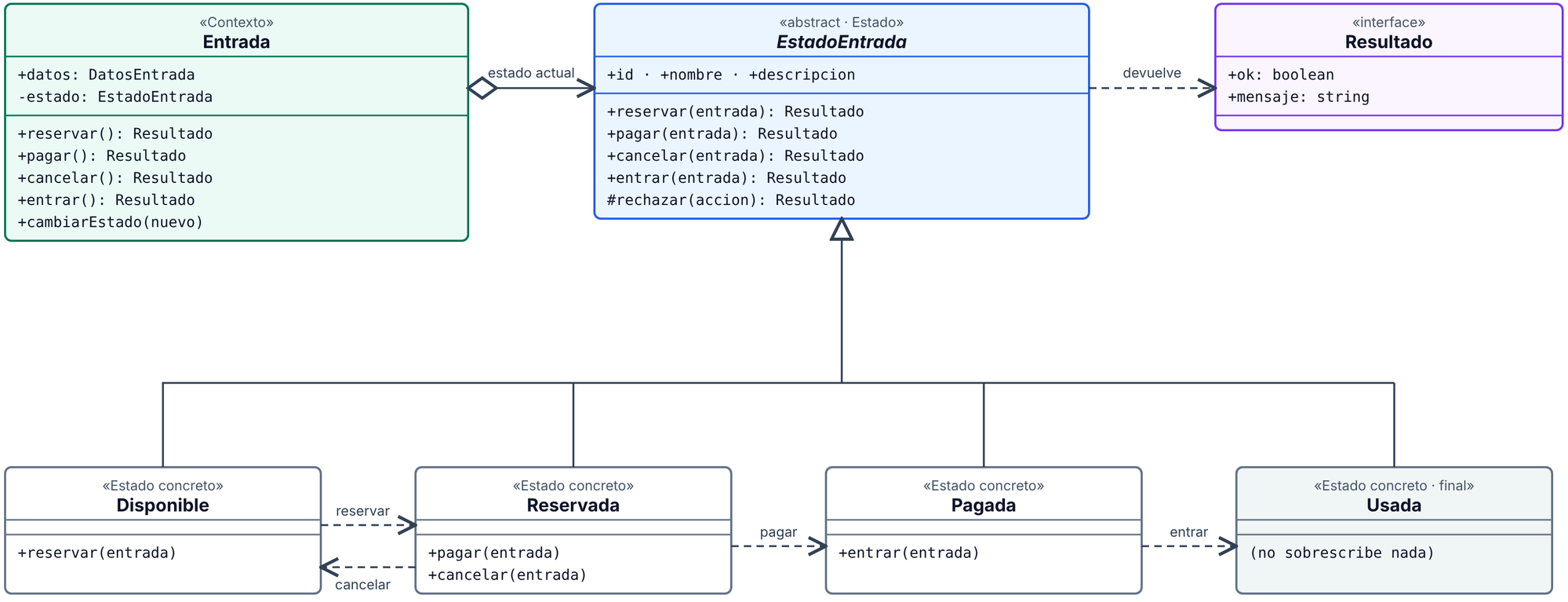


Las acciones que no aparecen como flecha se rechazan y la entrada se queda en el mismo estado.

Estado	reservar	pagar	cancelar	entrar
Disponible	→ Reservada	Rechaza	Rechaza	Rechaza
Reservada	Rechaza	→ Pagada	→ Disponible	Rechaza
Pagada	Rechaza	Rechaza	Rechaza (sin devoluciones)	→ Usada
Usada	Rechaza	Rechaza	Rechaza	Rechaza (ya se usó)

Ejemplo 1 · Diagrama de clases

Entrada = contexto · EstadoEntrada = estado




```
// Entrada (contexto): no tiene ningún "if" por estado, solo delega:
pagar() { return this.estado.pagar(this); }
```

```
// Lo que un estado no sobrescribe cae en la clase base:
rechazar(accion) → { ok: false, mensaje: "No se puede…" }
```

⇢ Línea punteada con flecha = el estado crea el siguiente y llama a entrada.cambiarEstado(...)

EJEMPLO 1 · EL CÓDIGO

Reservada sabe pagar y cancelar; nada más

Estado concreto

```
class Reservada extends EstadoEntrada {
  pagar(entrada) {
    entrada.cambiarEstado(new Pagada());
    return { ok: true, mensaje: 'Pago recibido' };
  }
  cancelar(entrada) {
    entrada.cambiarEstado(new Disponible());
    return { ok: true, mensaje: 'Reserva cancelada' };
  }
  // reservar() y entrar() se rechazan solos
}
```

Contexto

```
class Entrada {
  private estado = new Disponible();
  pagar() { return this.estado.pagar(this); }
  cambiarEstado(n) { this.estado = n; }
}
```

Ningún if por estado. Un estado nuevo es una clase nueva; las demás no se tocan.

Código real: src/app/state-simple/entrada.ts y estados/

EJEMPLO 2 · STATE + OBSERVER

El pedido de compra

State decide qué puede hacer el pedido en cada momento. Observer avisa a quien le interese cada vez que algo cambia.

State

5 estados: En carrito, Pendiente de pago, Pagado, Cancelado y Reembolsado.

Reglas propias: máximo 4 entradas, 3 pagos rechazados, reembolso con 7 días o más.

Observer

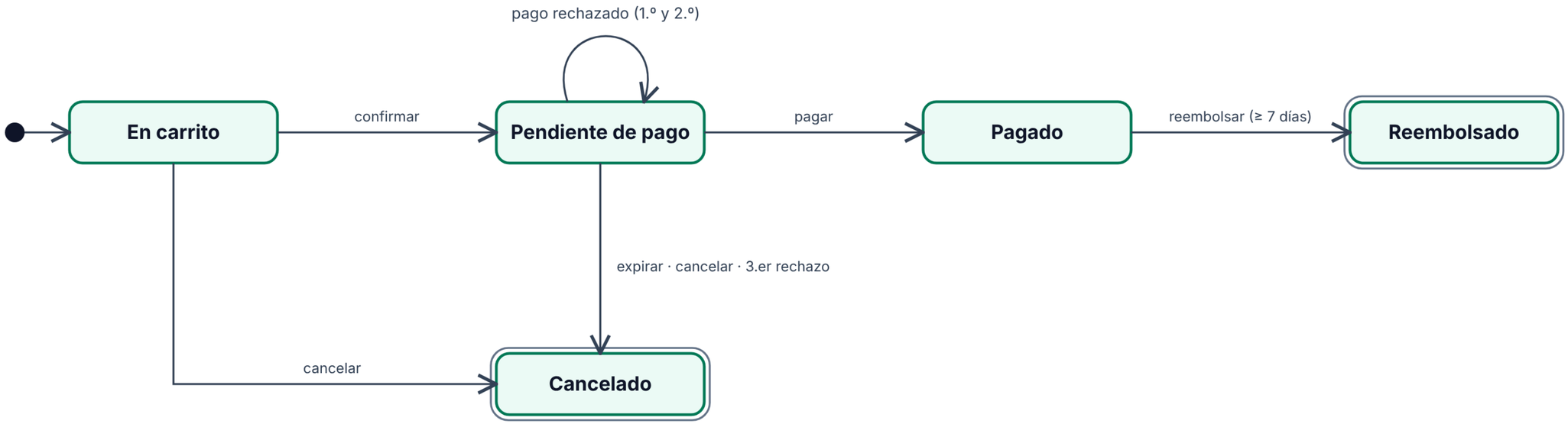
4 observadores: correo al cliente, panel de taquilla, auditoría y antifraude.

Se suscriben y se dan de baja sin que el pedido los conozca.

Ruta de la demo: /patronState/observer · código en src/app/state-observer

EJEMPLO 2 · DIAGRAMA DE ESTADOS

Un estado puede llevar cuenta

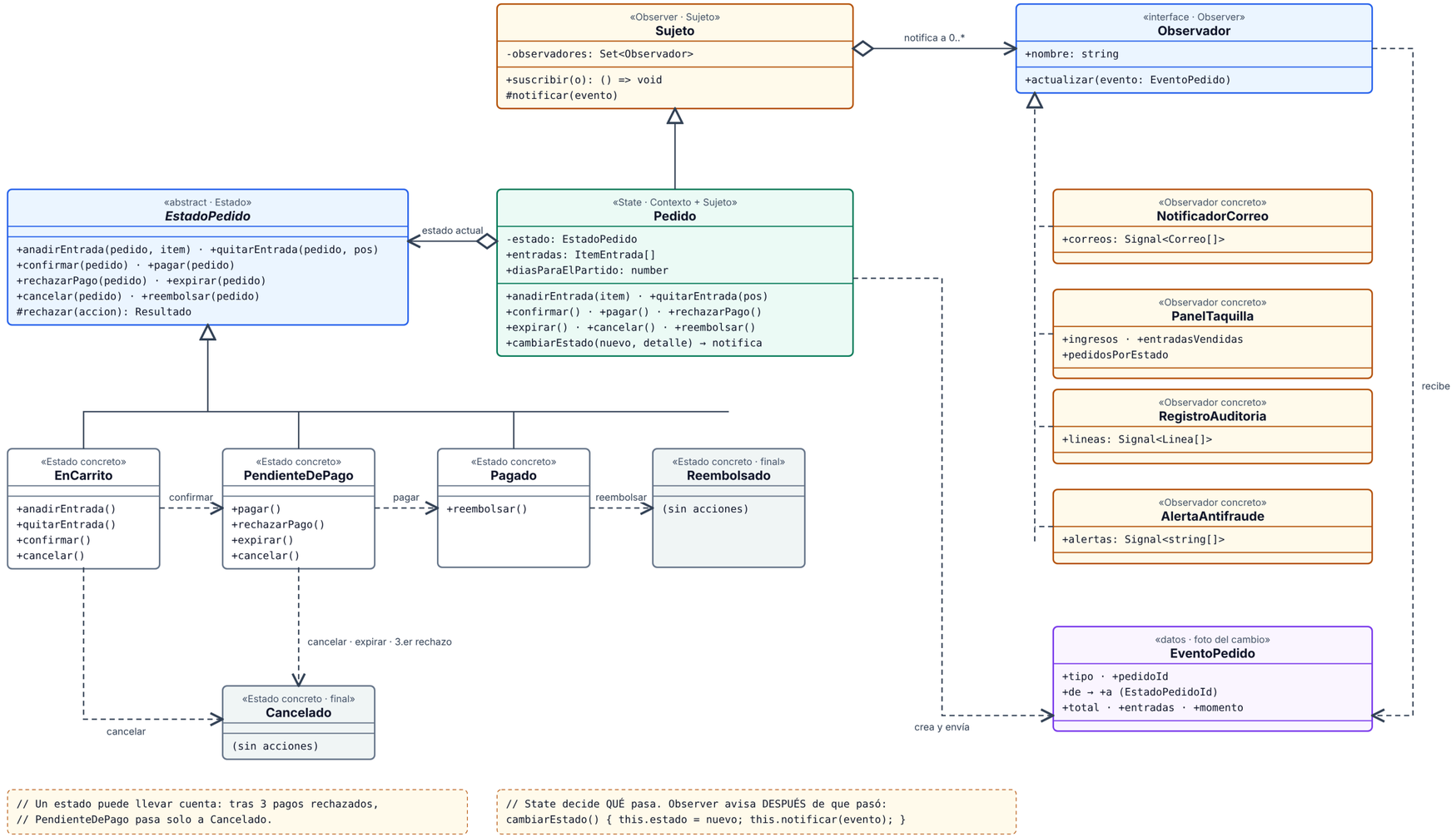


Pago rechazado
El 1.º y el 2.º dejan el pedido igual; el 3.º lo cancela.

Reembolso
Solo con 7 días o más antes del partido.

Ejemplo 2 · Diagrama de clases

Pedido es contexto de State y sujeto de Observer



Patrón State + Observer · Diagrama en alta resolución: docs/diagramas/03-ejemplo2-clases.svg

EJEMPLO 2 · CÓMO COLABORAN

Qué pasa al pulsar «Pagar»

1	La persona pide pagar	`pedido.pagar()`
2	State: el pedido delega en su estado	`estado.pagar(pedido)`
3	State: PendienteDePago pasa a Pagado	`cambiarEstado(new Pagado())`
4	Observer: el pedido avisa a sus suscriptores	`notificar(evento)`
5	Cada observador reacciona a su manera	Correo · taquilla suma los ingresos · auditoría anota · antifraude vigila

El estado no sabe que existen los observadores · los observadores no deciden nada

COMPARACIÓN

State frente a sus vecinos

Patrón	Qué resuelve	Cómo se diferencia
State	Cambiar de comportamiento según el estado	Los estados se conocen y se reemplazan entre sí
Strategy	Intercambiar un algoritmo	Las estrategias no se conocen; las elige el cliente
Observer	Avisar a varios objetos de un cambio	Los observadores se suscriben sin acoplarse al sujeto

En el ejemplo 2 se complementan: State gobierna qué puede pasar, Observer cuenta lo que ya pasó.

Fuente: refactoring.guru/es/design-patterns/state

EL PROYECTO

Ramas y demostración

`main`

Rama desplegada. Después del merge muestra State.

`flyweight`

Copia del código anterior: Flyweight y Bridge, tal como estaba.

`state`

Código nuevo: los dos ejemplos del patrón State.

Ejemplo 1

`/patronState/simple`

Ejemplo 2

`/patronState/observer`

Angular 19 · Tailwind CSS 4 · sin backend · 27 pruebas unitarias

CONCLUSIONES

State convierte los if en clases

- Cada estado es una clase con su propio comportamiento.
- El contexto solo delega: no pregunta en qué estado está.
- Con Observer, los cambios de estado avisan a otros sin acoplarse.

¿Preguntas?